Exercise 17 — Getting Started with FastAPI

Exercise Description

Use AI to get up to speed with FastAPI, from fundamentals to a working, structured API. The Part 4 challenge is to build a to-do list API supporting: create (title, description, due date), list with status filtering (completed/pending), mark complete, and delete — with validation, error handling, and automatic docs.

Files: - `app.py` — the FastAPI to-do application. - `test_app.py` — TestClient tests (6, all passing).

Part 1 — FastAPI Fundamentals (notes)

- **What it is:** a modern Python web framework for building APIs, built on Starlette (ASGI) and Pydantic. Async-capable and fast.
- **vs Flask/Django:** vs Flask it adds type-hint-driven validation and automatic OpenAPI docs out of the box; vs Django it's lighter (no ORM/admin), API-first.
- **Key advantages:** request/response validation from type hints, automatic interactive docs (`/docs` , `/redoc`), editor autocompletion, and async support.
- **Core terms:** *path operation* (a route + HTTP method via a decorator), *path/query parameters*, *Pydantic model* (typed request/response schema), *dependency injection* (`Depends`), *response_model*.

Part 2 — First API

A minimal app exposes `GET /` and, thanks to FastAPI, an auto-generated Swagger UI at `/docs`. Running locally: `uvicorn app:app --reload` .

Part 3 — Enhancements applied

- **Pydantic validation:** `TodoCreate` enforces `title` length (1-200), caps `description` , and types `due_date` as a `date` . Invalid input yields an automatic **422**.
- **Error handling:** a `_get_or_404` helper raises `HTTPException(404)` for unknown ids, reused by get/complete/delete.
- **Structure:** models, an in-memory store, a helper, and path operations are cleanly separated (a single file here; would split into `models/` , `routes/` for a larger app).
- **Correct status codes:** `201` on create, `204` on delete, `200` on complete.

Part 4 — To-Do CRUD challenge (delivered)

Method	Path	Behaviour
POST	`/todos`	create (title required, optional description/ due_date) → 201
GET	`/todos?status=pending|completed`	list, optional status filter
GET	`/todos/{id}`	fetch one (404 if missing)
PATCH	`/todos/{id}/complete`	mark completed
DELETE	`/todos/{id}`	delete → 204 (404 if missing)

Test run

```
6 passed in 0.84s
```

Covers: create, empty-title rejection (422), list + status filter, complete, delete, and 404s on missing ids. A `reset_store` fixture isolates each test.

Submission for Exercise 17 — Getting Started with FastAPI

1. FastAPI fundamentals notes: what it is, how it compares to Flask/Django, its key advantages (validation, auto docs, async), and core terminology — above.

2. First "Hello World" API + auto docs: `GET /` plus FastAPI's automatic `/docs`; run with `uvicorn app:app --reload`.

3. Enhanced API: Pydantic validation (422 on bad input), reusable `404` error handling, correct status codes, and separated structure — see `app.py`.

4. Completed to-do application: full CRUD (create with title/description/due date, list with pending/completed filter, mark complete, delete), verified by `test_app.py` — **6 tests passing**.